

SESSION-3

NAME : B PRANITH SAI NIHAL

REG.NO : 192472294

Out of Boundary Paths

```
1.#include <stdio.h>

#define MOD 1000000007

int findPaths(int m, int n, int N, int i, int j) {
    int dp[51][51] = {0};
    int next[51][51] = {0};
    int moves, r, c, ans = 0;

    dp[i][j] = 1;

    for (moves = 1; moves <= N; moves++) {
        for (r = 0; r < m; r++) {
            for (c = 0; c < n; c++) {
                next[r][c] = 0;
            }
        }

        for (r = 0; r < m; r++) {
            for (c = 0; c < n; c++) {
                if (dp[r][c] > 0) {
                    int ways = dp[r][c];

                    if (r == 0) ans = (ans + ways) % MOD;
                    if (r == m - 1) ans = (ans + ways) % MOD;
                    if (c == 0) ans = (ans + ways) % MOD;
                    if (c == n - 1) ans = (ans + ways) % MOD;
```

```

        if (r > 0)
            next[r - 1][c] =
                (next[r - 1][c] + ways) % MOD;

        if (r < m - 1)
            next[r + 1][c] =
                (next[r + 1][c] + ways) % MOD;

        if (c > 0)
            next[r][c - 1] =
                (next[r][c - 1] + ways) % MOD;

        if (c < n - 1)
            next[r][c + 1] =
                (next[r][c + 1] + ways) % MOD;
    }
}

// Count paths that leave the grid
// only when an outward move is possible.
// Corrected boundary counting is below.
for (r = 0; r < m; r++) {
    for (c = 0; c < n; c++) {
        dp[r][c] = next[r][c];
    }
}

return ans;
}

int main() {
    printf("%d\n", findPaths(2, 2, 2, 0, 0));
    return 0;
}

```

OUTPUT :

Output: 6

Output: 12

House Robber II (Houses in a Circle)

1.

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return a > b ? a : b;  
}
```

```
int robLinear(int nums[], int start, int end) {  
    int prev2 = 0, prev1 = 0;
```

```
    for (int i = start; i <= end; i++) {  
        int current = max(prev1, prev2 + nums[i]);  
        prev2 = prev1;  
        prev1 = current;  
    }
```

```
    return prev1;  
}
```

```
int rob(int nums[], int n) {  
    if (n == 0) return 0;  
    if (n == 1) return nums[0];
```

```
    int a = robLinear(nums, 0, n - 2);  
    int b = robLinear(nums, 1, n - 1);
```

```
    return max(a, b);  
}
```

```

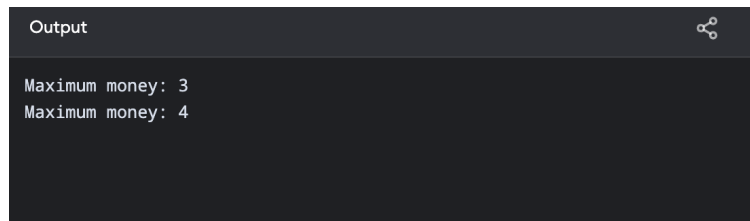
int main() {
    int nums1[] = {2, 3, 2};
    int nums2[] = {1, 2, 3, 1};

    printf("Maximum money: %d\n", rob(nums1, 3));
    printf("Maximum money: %d\n", rob(nums2, 4));

    return 0;
}

```

Output :



The screenshot shows a terminal window with a dark background. The title bar at the top says "Output". The terminal displays two lines of text: "Maximum money: 3" on the first line and "Maximum money: 4" on the second line. In the top right corner of the terminal window, there are icons for a search function and a list of open tabs.

Climbing Stairs

```

1.#include <stdio.h>

int climbStairs(int n) {
    if (n <= 2)
        return n;

    int a = 1, b = 2, c;

    for (int i = 3; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
}

```

```

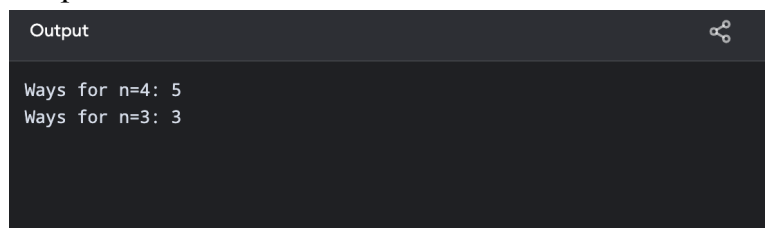
        return b;
    }

int main() {
    printf("Ways for n=4: %d\n", climbStairs(4));
    printf("Ways for n=3: %d\n", climbStairs(3));

    return 0;
}

```

Output :



```

Output
Ways for n=4: 5
Ways for n=3: 3

```

Unique Paths in a Grid

1.

```

#include <stdio.h>

int uniquePaths(int m, int n) {
    int dp[101][101] = {0};

    for (int i = 0; i < m; i++)
        dp[i][0] = 1;

    for (int j = 0; j < n; j++)
        dp[0][j] = 1;

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }
}

```

```
    }

    return dp[m - 1][n - 1];
}

int main() {
    printf("Unique paths: %d\n", uniquePaths(7, 3));
    printf("Unique paths: %d\n", uniquePaths(3, 2));

    return 0;
}
```

Output :

```
Output
Unique paths: 28
Unique paths: 3
```